Name :- Vrusha Patel

Enrolment Number :- 12402080601155

Batch :- 2D8

Subject :- Programming With Python (202044504)

Topic :- Assignment-2

GITHUB LINK: https://github.com/vrushapatel001-pixel/python-programming

1. Regex-Based Email Intelligence Extractor

```
# Q1: Regex-Based Email Intelligence Extractor

import re
import os

def extract_emails(file_path):
    # Regular expression for valid email addresses
    email_pattern = r'\b[A-Za-z0-9+._-]+@[A-Za-z0-9.-]+\.(?:com|edu|org)\b'

    domain_users = {}

    try:
        with open(file_path, "r", encoding="utf-8") as file:

            for line in file:
                # Find all valid emails in the current line
                emails = re.findall(email_pattern, line)

                for email in emails:
                    email = email.lower()

                    # Separate username and domain
                    username, domain = email.split("@", 1)

                    if domain not in domain_users:
                        domain_users[domain] = set()

                    # Store complete email to remove duplicates
                    domain_users[domain].add(email)

    except FileNotFoundError:
        print("File not found.")
        return

    except PermissionError:
        print("Permission denied.")
        return

    except Exception as e:
        print("Error while reading file:", e)
        return

    if not domain_users:
        print("\nNo valid email addresses found.")
        return

    print("\nEmail Analysis Result:")

    # Print domains in lexicographic order
    for domain in sorted(domain_users):

        emails = domain_users[domain]

        # Find lexicographically smallest email
        smallest_email = min(emails)
```

```
        print(
            domain,
            len(emails),
            smallest_email
        )

print("Regex-Based Email Intelligence Extractor")

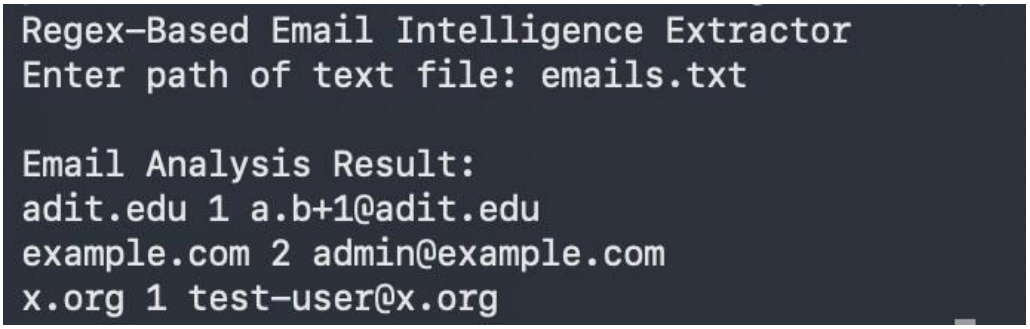
file_path = input(
    "Enter path of text file: "
).strip()

if file_path == "":
    print("File path cannot be empty.")

elif not os.path.isfile(file_path):
    print("The specified file does not exist.")

else:
    extract_emails(file_path)
```

Output :



2. Offline Web Scraping Ranker from HTML Pages

```
import re
import os

def extract_products(file_path):
    products = []

    try:
        with open(file_path, "r", encoding="utf-8") as file:
            html = file.read()

        # Extract product name, price and rating
        pattern = re.compile(
            r'<div class="product">\s*'
            r'<span  class="name">(.*?)</span>\s*'
            r'<span  class="price">(.*?)</span>\s*'
            r'<span class="rating">(.*?)</span>\s*'
            r'</div>',
            re.DOTALL | re.IGNORECASE
        )

        matches = pattern.findall(html)

        for name, price, rating in matches:
            name = re.sub(r"<.*?>", "", name).strip()
            price = re.sub(r"<.*?>", "", price).strip()
            rating = re.sub(r"<.*?>", "", rating).strip()

            # Normalize price
            price = re.sub(r"[^\d.]", "", price)

            try:
                price = float(price)
                rating = float(rating)
            except ValueError:
                continue

            products.append({
                "name": name,
                "price": price,
                "rating": rating
            })

    except FileNotFoundError:
        print("File not found:", file_path)
    except PermissionError:
        print("Permission denied:", file_path)
    except Exception as e:
        print("Error while reading file:", e)

    return products

def rank_products(products, k):
    # Remove duplicate products by name
    unique_products = {}
```

```
    for product in products:
        name = product["name"]

        if name not in unique_products:
            unique_products[name] = product
        else:
            # Keep the better product if duplicate name occurs
            old_product = unique_products[name]

            if (product["rating"], -product["price"], product["name"]) > \
                (old_product["rating"], -old_product["price"], old_product["name"]):
                unique_products[name] = product

    products = list(unique_products.values())

    # Highest rating first
    # If rating is same -> lower price first
    # If rating and price are same -> lexicographic name
    products.sort(
        key=lambda p: (-p["rating"], p["price"], p["name"])
    )

    return products[:k]

def main():
    print("Offline Web Scraping Ranker")

    try:
        first_line = input("Enter number of HTML pages and K: ").split()

        if len(first_line) != 2:
            print("Please enter exactly two values.")
            return

        p = int(first_line[0])
        k = int(first_line[1])

        if p <= 0 or k <= 0:
            print("Number of pages and K must be positive.")
            return

    except ValueError:
        print("Invalid input. Enter integers only.")
        return

    all_products = []

    for i in range(p):
        file_path = input(f"Enter path of HTML page {i + 1}: ").strip()

        if file_path == "":
            print("File path cannot be empty.")
            continue

        if not os.path.isfile(file_path):
            print("File does not exist:", file_path)
            continue
```

```
        products = extract_products(file_path)
        all_products.extend(products)

    if not all_products:
        print("\nNo valid products found.")
        return

    top_products = rank_products(all_products, k)

    print("\nTop Products:")
    for product in top_products:
        price = int(product["price"]) if product["price"].is_integer() else product["price"]

        print(
            product["name"],
            price,
            product["rating"]
        )

main()
```

Output :

```
Offline Web Scraping Ranker
Enter number of HTML pages and K: 2 2
Enter path of HTML page 1: page1.html
Enter path of HTML page 2: page2.html

Top Products:
Python Book 499 4.8
USB Keyboard 799 4.7
```

3. MySQL Student-Course Analytics Engine

```
import mysql.connector
import csv
import os

def connect_database():
    print("Enter MySQL Database Details")

    host = input("Enter host: ").strip()
    user = input("Enter username: ").strip()
    password = input("Enter password: ")
    database = input("Enter database name: ").strip()

    try:
        connection = mysql.connector.connect(
            host=host,
            user=user,
            password=password,
            database=database
        )

        print("Database connected successfully.")
        return connection

    except mysql.connector.Error as e:
        print("Database connection error:", e)
        return None

def create_tables(connection):
    try:
        cursor = connection.cursor()

        student_table = """
        CREATE TABLE IF NOT EXISTS Student (
            student_id INT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            spi DECIMAL(4,2) NOT NULL
        )
        """

        registration_table = """
        CREATE TABLE IF NOT EXISTS CourseRegistration (
            student_id INT,
            course_id VARCHAR(20),
            FOREIGN KEY (student_id)
                REFERENCES Student(student_id)
        )
        """

        cursor.execute(student_table)
        cursor.execute(registration_table)

        connection.commit()
        cursor.close()

        print("Tables created successfully.")
```

```
    except mysql.connector.Error as e:
        print("Error creating tables:", e)

def insert_students(connection, file_path):
    if not os.path.isfile(file_path):
        print("Student CSV file not found.")
        return

    try:
        cursor = connection.cursor()

        with open(file_path, "r", encoding="utf-8") as file:
            reader = csv.DictReader(file)

            for row in reader:
                try:
                    student_id = int(row["student_id"])
                    name = row["name"].strip()
                    spi = float(row["spi"])

                    if spi < 0 or spi > 10:
                        print("Invalid SPI for student:", student_id)
                        continue

                    query = """
                    INSERT INTO Student (student_id, name, spi)
                    VALUES (%s, %s, %s)
                    ON DUPLICATE KEY UPDATE
                    name = VALUES(name),
                    spi = VALUES(spi)
                    """

                    cursor.execute(
                        query,
                        (student_id, name, spi)
                    )

                except (ValueError, KeyError):
                    print("Invalid student record:", row)

        connection.commit()
        cursor.close()

        print("Student records inserted successfully.")

    except (OSError, mysql.connector.Error) as e:
        print("Error reading student CSV:", e)

def insert_registrations(connection, file_path):
    if not os.path.isfile(file_path):
        print("Registration CSV file not found.")
        return

    try:
        cursor = connection.cursor()
```

```
        with open(file_path, "r", encoding="utf-8") as file:
            reader = csv.DictReader(file)

            for row in reader:
                try:
                    student_id = int(row["student_id"])
                    course_id = row["course_id"].strip()

                    query = """
                    INSERT INTO CourseRegistration
                    (student_id, course_id)
                    VALUES (%s, %s)
                    """

                    cursor.execute(
                        query,
                        (student_id, course_id)
                    )

                except (ValueError, KeyError):
                    print("Invalid registration record:", row)

        connection.commit()
        cursor.close()

        print("Course registration records inserted successfully.")

    except (OSError, mysql.connector.Error) as e:
        print("Error reading registration CSV:", e)

def display_students(connection, course_id, spi_threshold):
    try:
        cursor = connection.cursor()

        query = """
        SELECT s.student_id, s.name, s.spi, cr.course_id
        FROM Student s
        INNER JOIN CourseRegistration cr
            ON s.student_id = cr.student_id
        WHERE cr.course_id = %s
        AND s.spi > %s
        ORDER BY s.spi DESC, s.student_id ASC
        """

        cursor.execute(
            query,
            (course_id, spi_threshold)
        )

        results = cursor.fetchall()

        print("\nStudents Registered for Course:", course_id)
        print("SPI greater than:", spi_threshold)

        if not results:
            print("No matching students found.")
        else:
```

```
            print("\nStudent ID  Name      SPI   Course ID")
            print(" ------------------------------------ ")

            for student_id, name, spi, course in results:
                print(
                    f"{student_id:<11} "
                    f"{name:<11} "
                    f"{spi:<6} "
                    f"{course}"
                )

        cursor.close()

    except mysql.connector.Error as e:
        print("Error executing query:", e)

def main():
    print("MySQL Student-Course Analytics Engine")

    connection = connect_database()

    if connection is None:
        return

    create_tables(connection)

    student_file = input(
        "Enter path of student.csv: "
    ).strip()

    registration_file = input(
        "Enter path of registration.csv: "
    ).strip()

    insert_students(connection, student_file)
    insert_registrations(connection, registration_file)

    course_id = input(
        "Enter course ID: "
    ).strip()

    try:
        spi_threshold = float(
            input("Enter SPI threshold: ")
        )

        if spi_threshold < 0 or spi_threshold > 10:
            print("SPI must be between 0 and 10.")
            connection.close()
            return

    except ValueError:
        print("Invalid SPI threshold.")
        connection.close()
        return

    display_students(
```

```
        connection,
        course_id,
        spi_threshold
    )

    connection.close()
    print("\nDatabase connection closed.")

main()
```

Output :

```
MySQL Student-Course Analytics Engine

Enter MySQL Database Details
Enter host: localhost
Enter username: root
Enter password: 1234
Enter database name: college

Database connected successfully.
Tables created successfully.

Enter path of student.csv: student.csv
Enter path of registration.csv: registration.csv
Student records inserted successfully.
Course registration records inserted successfully.

Enter course ID: PY101
Enter SPI threshold: 8.0

Students Registered for Course: PY101
SPI greater than: 8.0

Student ID  Name        SPI    Course ID
----------------------------------------
2201        Asha        9.1    PY101
2207        Dev         8.4    PY101

Database connection closed.
```

4. SQL Join Query Builder and Validator

```
import mysql.connector

# Allowed columns for SELECT
ALLOWED_COLUMNS = {
    "student.id": "s.student_id",
    "student.name": "s.name",
    "student.spi": "s.spi",
    "course.id": "c.id",
    "course.name": "c.name"
}

# Allowed columns for WHERE
ALLOWED_FILTER_COLUMNS = {
    "student.id": "s.student_id",
    "student.name": "s.name",
    "student.spi": "s.spi",
    "course.id": "c.id",
    "course.name": "c.name"
}

# Allowed columns for ORDER BY
ALLOWED_ORDER_COLUMNS = {
    "student.id": "s.student_id",
    "student.name": "s.name",
    "student.spi": "s.spi",
    "course.id": "c.id",
    "course.name": "c.name"
}

def connect_database():
    print("Enter MySQL Database Details")

    host = input("Enter host: ").strip()
    user = input("Enter username: ").strip()
    password = input("Enter password: ")
    database = input("Enter database name: ").strip()

    try:
        connection = mysql.connector.connect(
            host=host,
            user=user,
            password=password,
            database=database
        )

        print("Database connected successfully.")
        return connection

    except mysql.connector.Error as e:
        print("Database connection error:", e)
        return None

def create_tables(connection):
    try:
        cursor = connection.cursor()
```

```
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS Student (
                student_id INT PRIMARY KEY,
                name VARCHAR(100),
                spi DECIMAL(4,2)
            )
        """)

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS Course (
                id VARCHAR(20) PRIMARY KEY,
                name VARCHAR(100)
            )
        """)

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS CourseRegistration (
                student_id INT,
                course_id VARCHAR(20),
                FOREIGN KEY (student_id)
                    REFERENCES Student(student_id),
                FOREIGN KEY (course_id)
                    REFERENCES Course(id)
            )
        """)

        connection.commit()
        cursor.close()

        print("Required tables are ready.")

    except mysql.connector.Error as e:
        print("Table creation error:", e)

def get_columns():
    columns_input = input(
        "Enter columns separated by comma "
        "(example: student.name, course.name): "
    ).strip()

    columns = [column.strip() for column in columns_input.split(",")]

    if not columns or any(column not in ALLOWED_COLUMNS for column in columns):
        print("Invalid column detected.")
        return None

    return columns

def get_filter():
    use_filter = input(
        "Do you want to add a WHERE condition? (yes/no): "
    ).strip().lower()

    if use_filter == "no":
        return "", []
```

```
    if use_filter != "yes":
        print("Invalid choice.")
        return None, None

    column = input(
        "Enter filter column "
        "(example: course.id): "
    ).strip()

    if column not in ALLOWED_FILTER_COLUMNS:
        print("Invalid filter column.")
        return None, None

    operator = input(
        "Enter operator (=, >, <, >=, <=, LIKE): "
    ).strip().upper()

    allowed_operators = ["=", ">", "<", ">=", "<=", "LIKE"]

    if operator not in allowed_operators:
        print("Invalid operator.")
        return None, None

    value = input("Enter filter value: ").strip()

    sql_column = ALLOWED_FILTER_COLUMNS[column]

    condition = f"{sql_column} {operator} %s"

    return " WHERE " + condition, [value]

def get_order():
    order_column = input(
        "Enter order column (or press Enter to skip): "
    ).strip()

    if order_column == "":
        return ""

    if order_column not in ALLOWED_ORDER_COLUMNS:
        print("Invalid order column.")
        return None

    direction = input(
        "Enter sort direction (ASC/DESC): "
    ).strip().upper()

    if direction not in ["ASC", "DESC"]:
        print("Invalid sort direction.")
        return None

    return (
        " ORDER BY "
        + ALLOWED_ORDER_COLUMNS[order_column]
        + " "
        + direction
    )
```

```
def get_limit():
    try:
        limit = int(input("Enter LIMIT (1-1000): "))

        if limit < 1 or limit > 1000:
            print("LIMIT must be between 1 and 1000.")
            return None

        return limit

    except ValueError:
        print("LIMIT must be an integer.")
        return None

def build_and_execute_query(connection):
    columns = get_columns()

    if columns is None:
        return

    where_clause, parameters = get_filter()

    if where_clause is None:
        return

    order_clause = get_order()

    if order_clause is None:
        return

    limit = get_limit()

    if limit is None:
        return

    selected_columns = ", ".join(
        ALLOWED_COLUMNS[column]
        for column in columns
    )

    query = f"""
        SELECT {selected_columns}
        FROM Student s
        INNER JOIN CourseRegistration cr
            ON s.student_id = cr.student_id
        INNER JOIN Course c
            ON cr.course_id = c.id
        {where_clause}
        {order_clause}
        LIMIT %s
    """

    parameters.append(limit)

    print("\nSQL_OK")
    print("Generated safe SQL template:")
```

```
    print(query.strip())
    print("Parameters:", parameters)

    try:
        cursor = connection.cursor()
        cursor.execute(query, tuple(parameters))

        results = cursor.fetchall()

        print("\nQuery Result:")

        if not results:
            print("No matching records found.")
        else:
            for row in results:
                print(*row)

        cursor.close()

    except mysql.connector.Error as e:
        print("Query execution error:", e)

def main():
    print("SQL Join Query Builder and Validator")

    connection = connect_database()

    if connection is None:
        return

    create_tables(connection)

    build_and_execute_query(connection)

    connection.close()

    print("\nDatabase connection closed.")

main()
```

Output :

```
SQL Join Query Builder and Validator

Enter MySQL Database Details
Enter host: localhost
Enter username: root
Enter password: 1234
Enter database name: college

Database connected successfully.
Required tables are ready.

Enter columns separated by comma (example: student.name, course.name): student.name, course.nam
Do you want to add a WHERE condition? (yes/no): yes
Enter filter column (example: course.id): course.id
Enter operator (=, >, <, >=, <=, LIKE): =
Enter filter value: PY101
Enter order column (or press Enter to skip): student.spi
Enter sort direction (ASC/DESC): DESC
Enter LIMIT (1-1000): 2

SQL_OK
Generated safe SQL template:
SELECT s.name, c.name
FROM Student s
INNER JOIN CourseRegistration cr
    ON s.student_id = cr.student_id
INNER JOIN Course c
    ON cr.course_id = c.id
WHERE c.id = %s
ORDER BY s.spi DESC
LIMIT %s

Parameters: ['PY101', 2]

Query Result:
Asha Programming with Python
Dev Programming with Python

Database connection closed.
```

5. Tkinter Contact Manager with MySQL Backend

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector
import re

class ContactManager:

    def __init__(self, root):
        self.root = root
        self.root.title("Contact Manager")
        self.root.geometry("850x650")

        self.connection = None
        self.selected_id = None
        self.contact_ids = []

        # Variables
        self.name_var = tk.StringVar()
        self.email_var = tk.StringVar()
        self.phone_var = tk.StringVar()
        self.category_var = tk.StringVar(value="Friend")
        self.search_var = tk.StringVar()
        self.sort_var = tk.StringVar(value="name")
        self.show_all_var = tk.BooleanVar(value=True)

        self.create_menu()
        self.create_widgets()

    def create_menu(self):
        menu_bar = tk.Menu(self.root)

        file_menu = tk.Menu(menu_bar, tearoff=0)
        file_menu.add_command(label="Connect Database",
                              command=self.connect_database)
        file_menu.add_command(label="Exit",
                              command=self.root.destroy)

        menu_bar.add_cascade(label="File", menu=file_menu)

        help_menu = tk.Menu(menu_bar, tearoff=0)
        help_menu.add_command(
            label="About",
            command=lambda: messagebox.showinfo(
                "About",
                "Tkinter Contact Manager\nPython Assignment 2"
            )
        )

        menu_bar.add_cascade(label="Help", menu=help_menu)

        self.root.config(menu=menu_bar)

    def create_widgets(self):

        title = tk.Label(
```

```
        self.root,
        text="CONTACT MANAGER",
        font=("Arial", 20, "bold")
    )
    title.pack(pady=10)

    form_frame = tk.Frame(self.root)
    form_frame.pack(pady=5)

    # Name
    tk.Label(form_frame, text="Name:").grid(
        row=0, column=0, padx=5, pady=5
    )

    tk.Entry(
        form_frame,
        textvariable=self.name_var,
        width=30
    ).grid(row=0, column=1, padx=5)

    # Email
    tk.Label(form_frame, text="Email:").grid(
        row=1, column=0, padx=5, pady=5
    )

    tk.Entry(
        form_frame,
        textvariable=self.email_var,
        width=30
    ).grid(row=1, column=1, padx=5)

    # Phone
    tk.Label(form_frame, text="Phone:").grid(
        row=2, column=0, padx=5, pady=5
    )

    tk.Entry(
        form_frame,
        textvariable=self.phone_var,
        width=30
    ).grid(row=2, column=1, padx=5)

    # Category - Combobox
    tk.Label(form_frame, text="Category:").grid(
        row=3, column=0, padx=5, pady=5
    )

    category_box = ttk.Combobox(
        form_frame,
        textvariable=self.category_var,
        values=[
            "Friend",
            "Family",
            "Faculty",
            "Student",
            "Work",
            "Other"
```

```
        ],
        state="readonly",
        width=27
    )
    category_box.grid(row=3, column=1, padx=5)

    # Notes - Text
    tk.Label(form_frame, text="Notes:").grid(
        row=4, column=0, padx=5, pady=5
    )

    self.notes_text = tk.Text(
        form_frame,
        width=30,
        height=4
    )
    self.notes_text.grid(row=4, column=1, padx=5)

    # Buttons
    button_frame = tk.Frame(self.root)
    button_frame.pack(pady=10)

    tk.Button(
        button_frame,
        text="Connect Database",
        command=self.connect_database
    ).grid(row=0, column=0, padx=5)

    tk.Button(
        button_frame,
        text="Add",
        command=self.add_contact
    ).grid(row=0, column=1, padx=5)

    tk.Button(
        button_frame,
        text="Update",
         command=self.update_contact
    ).grid(row=0, column=2, padx=5)

    tk.Button(
        button_frame,
        text="Delete",
         command=self.delete_contact
    ).grid(row=0, column=3, padx=5)

    tk.Button(
        button_frame,
        text="Clear",
        command=self.clear_fields
    ).grid(row=0, column=4, padx=5)

    # Search
    search_frame = tk.Frame(self.root)
    search_frame.pack(pady=5)

    tk.Label(
```

```
        search_frame,
        text="Search Name/Email/Phone:"
    ).pack(side=tk.LEFT)

    tk.Entry(
        search_frame,
        textvariable=self.search_var,
        width=25
    ).pack(side=tk.LEFT, padx=5)

    tk.Button(
        search_frame,
        text="Search",
        command=self.search_contacts
    ).pack(side=tk.LEFT, padx=5)

    tk.Button(
        search_frame,
        text="Show All",
        command=self.load_contacts
    ).pack(side=tk.LEFT, padx=5)

    # Radio buttons for sorting
    sort_frame = tk.Frame(self.root)
    sort_frame.pack(pady=5)

    tk.Label(
        sort_frame,
        text="Sort By:"
    ).pack(side=tk.LEFT)

    tk.Radiobutton(
        sort_frame,
        text="Name",
        variable=self.sort_var,
        value="name",
        command=self.load_contacts
    ).pack(side=tk.LEFT)

    tk.Radiobutton(
        sort_frame,
        text="Email",
        variable=self.sort_var,
        value="email",
        command=self.load_contacts
    ).pack(side=tk.LEFT)

    tk.Radiobutton(
        sort_frame,
        text="Category",
        variable=self.sort_var,
        value="category",
        command=self.load_contacts
    ).pack(side=tk.LEFT)

    # Checkbutton
    tk.Checkbutton(
```

```
            sort_frame,
            text="Show All Contacts",
            variable=self.show_all_var,
            command=self.load_contacts
        ).pack(side=tk.LEFT, padx=15)

        # Listbox
        tk.Label(
            self.root,
            text="Contact List",
            font=("Arial", 12, "bold")
        ).pack(pady=5)

        list_frame = tk.Frame(self.root)
        list_frame.pack()

        scrollbar = tk.Scrollbar(list_frame)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        self.contact_list = tk.Listbox(
            list_frame,
            width=100,
            height=12,
            yscrollcommand=scrollbar.set
        )
        self.contact_list.pack(side=tk.LEFT)

        scrollbar.config(command=self.contact_list.yview)

        self.contact_list.bind(
            "<<ListboxSelect>>",
            self.select_contact
        )

        self.status_label = tk.Label(
            self.root,
            text="Not connected to database",
            fg="red"
        )
        self.status_label.pack(pady=10)

    def connect_database(self):

        try:
            self.connection = mysql.connector.connect(
                host="localhost",
                user="root",
                password="1234",
                database="college"
            )

            cursor = self.connection.cursor()

            cursor.execute("""
                CREATE TABLE IF NOT EXISTS Contact (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    name VARCHAR(100) NOT NULL,
```

```
                email VARCHAR(100) UNIQUE NOT NULL,
                phone VARCHAR(20),
                category VARCHAR(50),
                notes TEXT
            )
        """)

        self.connection.commit()
        cursor.close()

        self.status_label.config(
            text="Database connected successfully",
            fg="green"
        )

        messagebox.showinfo(
            "Success",
            "Database connected successfully."
        )

        self.load_contacts()

    except mysql.connector.Error as e:
        messagebox.showerror(
            "Database Error",
            "Could not connect to MySQL.\n\n" + str(e)
        )

def validate_email(self, email):

    pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'

    return re.match(pattern, email) is not None

def get_form_data(self):

    name  =  self.name_var.get().strip()
    email  =  self.email_var.get().strip()
    phone = self.phone_var.get().strip()
    category = self.category_var.get().strip()
    notes = self.notes_text.get("1.0", tk.END).strip()

    if name == "":
        messagebox.showwarning(
            "Validation",
            "Name cannot be empty."
        )
        return None

    if not self.validate_email(email):
        messagebox.showwarning(
            "Validation",
            "Enter a valid email address."
        )
        return None

    if phone == "":
```

```
            messagebox.showwarning(
                "Validation",
                "Phone cannot be empty."
            )
            return None

        return name, email, phone, category, notes

    def add_contact(self):

        if self.connection is None:
            messagebox.showwarning(
                "Database",
                "Please connect to database first."
            )
            return

        data = self.get_form_data()

        if data is None:
            return

        try:
            cursor = self.connection.cursor()

            query = """
                INSERT INTO Contact
                (name, email, phone, category, notes)
                VALUES (%s, %s, %s, %s, %s)
            """

            cursor.execute(query, data)
            self.connection.commit()

            cursor.close()

            messagebox.showinfo(
                "Success",
                "Record inserted successfully."
            )

            self.clear_fields()
            self.load_contacts()

        except mysql.connector.IntegrityError:
            messagebox.showerror(
                "Duplicate Email",
                "This email already exists."
            )

        except mysql.connector.Error as e:
            messagebox.showerror(
                "Database Error",
                str(e)
            )

    def update_contact(self):
```

```
    if self.connection is None or self.selected_id is None:
        messagebox.showwarning(
            "Update",
            "Select a contact first."
        )
        return

    data = self.get_form_data()

    if data is None:
        return

    try:
        cursor = self.connection.cursor()

        query = """
            UPDATE Contact
            SET name=%s,
                email=%s,
                phone=%s,
                category=%s,
                notes=%s
            WHERE id=%s
        """

        cursor.execute(
            query,
            data + (self.selected_id,)
        )

        self.connection.commit()
        cursor.close()

        messagebox.showinfo(
            "Success",
            "Record updated successfully."
        )

        self.clear_fields()
        self.load_contacts()

    except mysql.connector.IntegrityError:
        messagebox.showerror(
            "Duplicate Email",
            "This email already exists."
        )

    except mysql.connector.Error as e:
        messagebox.showerror(
            "Database Error",
            str(e)
        )

def delete_contact(self):

    if self.connection is None or self.selected_id is None:
```

```
            messagebox.showwarning(
                "Delete",
                "Select a contact first."
            )
            return

        try:
            cursor = self.connection.cursor()

            cursor.execute(
                "DELETE FROM Contact WHERE id=%s",
                (self.selected_id,)
            )

            self.connection.commit()
            cursor.close()

            messagebox.showinfo(
                "Success",
                "Record deleted successfully."
            )

            self.clear_fields()
            self.load_contacts()

        except mysql.connector.Error as e:
            messagebox.showerror(
                "Database Error",
                str(e)
            )

def load_contacts(self):

    if self.connection is None:
        return

    try:
        cursor = self.connection.cursor()

        sort_column = self.sort_var.get()

        if sort_column not in ["name", "email", "category"]:
            sort_column = "name"

        query = f"""
            SELECT id, name, email, phone, category
            FROM Contact
            ORDER BY {sort_column}
        """

        cursor.execute(query)
        rows = cursor.fetchall()

        self.display_contacts(rows)

        cursor.close()
```

```
        except mysql.connector.Error as e:
            messagebox.showerror(
                "Database Error",
                str(e)
            )

    def search_contacts(self):

        if self.connection is None:
            messagebox.showwarning(
                "Database",
                "Please connect to database first."
            )
            return

        search = self.search_var.get().strip()

        if search == "":
            self.load_contacts()
            return

        try:
            cursor = self.connection.cursor()

            query = """
                SELECT id, name, email, phone, category
                FROM Contact
                WHERE name LIKE %s
                   OR email LIKE %s
                   OR phone LIKE %s
                ORDER BY name
            """

            value = "%" + search + "%"

            cursor.execute(
                query,
                (value, value, value)
            )

            rows = cursor.fetchall()

            self.display_contacts(rows)

            cursor.close()

        except mysql.connector.Error as e:
            messagebox.showerror(
                "Database Error",
                str(e)
            )

    def display_contacts(self, rows):

        self.contact_list.delete(0, tk.END)
        self.contact_ids.clear()
```

```
    for row in rows:
        contact_id, name, email, phone, category = row

        display = (
            f"{contact_id} | "
            f"{name} | "
            f"{email} | "
            f"{phone} | "
            f"{category}"
        )

        self.contact_list.insert(tk.END, display)
        self.contact_ids.append(contact_id)

def select_contact(self, event):

    selection = self.contact_list.curselection()

    if not selection:
        return

    index = selection[0]

    if index >= len(self.contact_ids):
        return

    self.selected_id = self.contact_ids[index]

    try:
        cursor = self.connection.cursor()

        cursor.execute(
            """
            SELECT name, email, phone, category, notes
            FROM Contact
            WHERE id=%s
            """,
            (self.selected_id,)
        )

        row = cursor.fetchone()
        cursor.close()

        if row:
            self.name_var.set(row[0])
            self.email_var.set(row[1])
            self.phone_var.set(row[2])
            self.category_var.set(row[3])

            self.notes_text.delete("1.0", tk.END)
            self.notes_text.insert("1.0", row[4] or "")

    except mysql.connector.Error as e:
        messagebox.showerror(
            "Database Error",
            str(e)
        )
```

```
    def clear_fields(self):

        self.name_var.set("")
        self.email_var.set("")
        self.phone_var.set("")
        self.category_var.set("Friend")

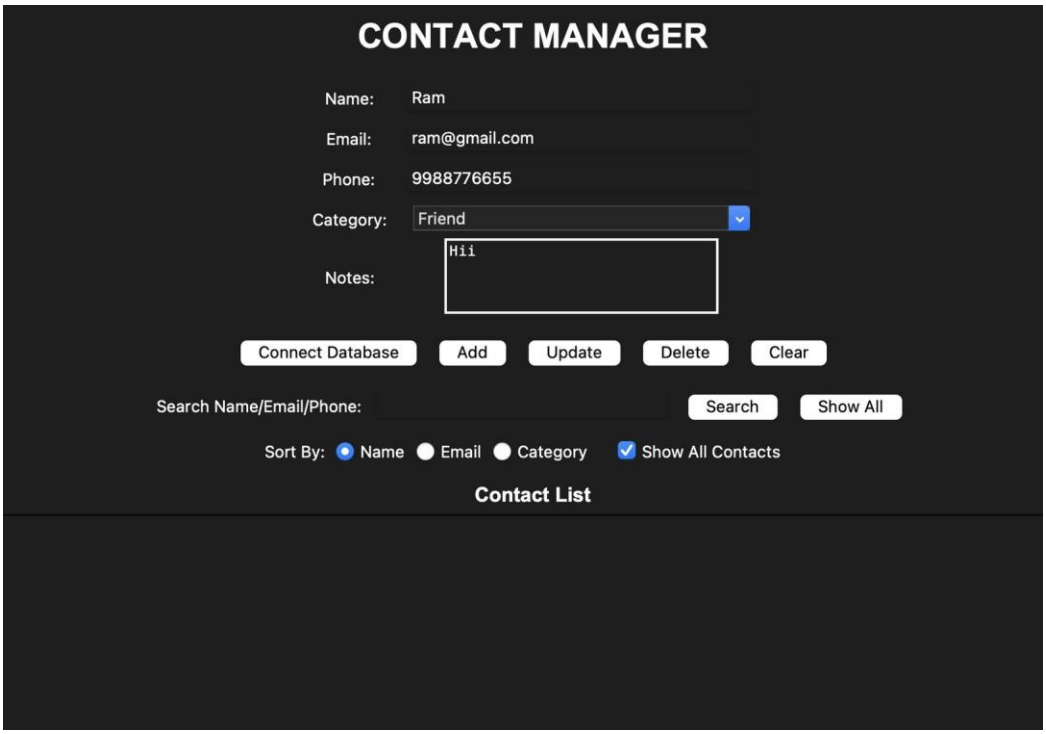
        self.notes_text.delete("1.0", tk.END)

        self.selected_id = None

        self.contact_list.selection_clear(0, tk.END)

root = tk.Tk()
app = ContactManager(root)
root.mainloop()
```

Output :



6. Log Anomaly Detector using Sliding Window

```
import re
from collections import defaultdict, deque

def time_to_minutes(time_string):
    """Convert HH:MM into minutes."""
    hour, minute = map(int, time_string.split(":"))
    return hour * 60 + minute

def detect_anomalies(x, t, logs):
    # Store failed login attempts for every user
    failed_attempts = defaultdict(deque)

    suspicious_users = {}

    # Regex for parsing log lines
    pattern = re.compile(
        r"^(\d{2}:\d{2})\s+(\S+)\s+(\S+)\s+(FAIL|SUCCESS)$"
    )

    for log in logs:

        match = pattern.match(log.strip())

        if not match:
            print("Invalid log ignored:", log)
            continue

        timestamp, user, ip, status = match.groups()

        current_time = time_to_minutes(timestamp)

        if status == "FAIL":

            # Add current failed attempt
            failed_attempts[user].append(
                (current_time, ip)
            )

        else:
            # Remove failures outside the T-minute window
            while (
                failed_attempts[user]
                and current_time - failed_attempts[user][0][0] > t
            ):
                failed_attempts[user].popleft()

            failures = failed_attempts[user]

            # Check whether at least X failures exist
            if len(failures) >= x:

                # Check if successful login is from a different IP
                different_ip = any(
                    failed_ip != ip
                    for _, failed_ip in failures
```

```
                )

                if different_ip and user not in suspicious_users:
                    suspicious_users[user] = timestamp

    return suspicious_users

def main():

    print("Log Anomaly Detector using Sliding Window")

    try:
        first_input = input(
            "Enter X and T (failed attempts and time window): "
        ).split()

        if len(first_input) != 2:
            print("Enter exactly two values.")
            return

        x = int(first_input[0])
        t = int(first_input[1])

        if x <= 0 or t <= 0:
            print("X and T must be positive.")
            return

        n = int(input("Enter number of log entries: "))

        if n <= 0:
            print("Number of log entries must be positive.")
            return

    except ValueError:
        print("Invalid input. Enter integers only.")
        return

    logs = []

    print("Enter log entries:")

    for i in range(n):
        log = input(f"Log {i + 1}: ").strip()
        logs.append(log)

    suspicious_users = detect_anomalies(
        x,
        t,
        logs
    )

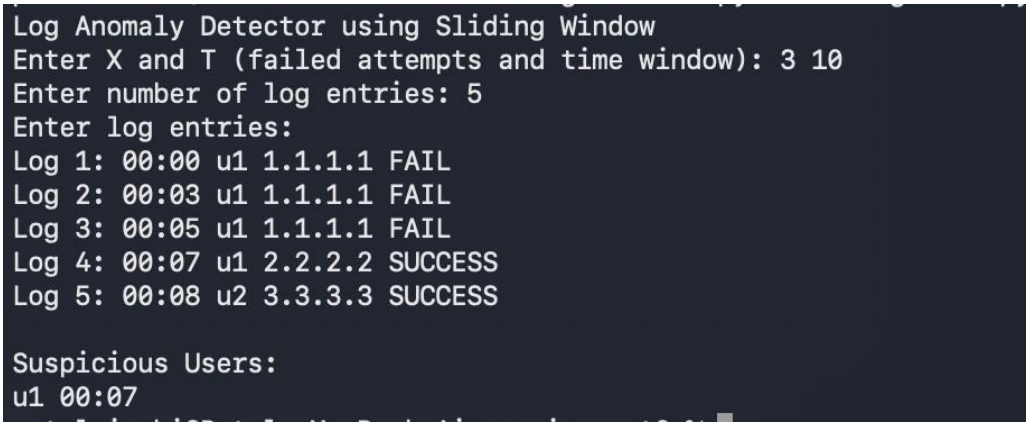
    print("\nSuspicious Users:")

    if not suspicious_users:
        print("No suspicious users found.")
        return
```

```
    for user in sorted(suspicious_users):
        print(
            user,
            suspicious_users[user]
        )

main()
```

Output :



7. Generator-Based JSON ETL Pipeline

```
import json
import os

def read_jsonl(file_path):
    """
    Generator function that reads one JSON record at a time.
    The complete file is not loaded into memory.
    """

    try:
        with open(file_path, "r", encoding="utf-8") as file:

            for line_number, line in enumerate(file, start=1):

                line = line.strip()

                if line == "":
                    continue

                try:
                    record = json.loads(line)
                    yield record

                except json.JSONDecodeError:
                    yield {
                        "_corrupted": True,
                        "_line": line_number
                    }

    except FileNotFoundError:
        print("File not found.")
    except PermissionError:
        print("Permission denied.")

def transform_temperature(value):
    """
    Convert temperature into a numeric Celsius value.
    """

    try:
        temperature = float(value)

        if temperature != temperature:
            return None

        return temperature

    except (ValueError, TypeError):
        return None

def process_records(file_path):
    """
    Process sensor records and generate statistics for each device.
    """
```

```
device_data = {}

for record in read_jsonl(file_path):

    # Handle corrupted JSON records
    if record.get("_corrupted", False):
        continue

    device_id = record.get("device_id")
    temperature = record.get("temperature_c")
    humidity = record.get("humidity")

    # Validate required fields
    if (
        device_id is None
        or temperature is None
        or humidity is None
    ):
        if device_id is not None:
            if device_id not in device_data:
                device_data[device_id] = {
                    "count": 0,
                    "temperatures": [],
                    "corrupted": 0
                }

            device_data[device_id]["corrupted"] += 1

        continue

    # Transform temperature
    temperature = transform_temperature(temperature)

    # Validate humidity
    try:
        humidity = float(humidity)

        if humidity < 0 or humidity > 100:
            raise ValueError

    except (ValueError, TypeError):
        if device_id not in device_data:
            device_data[device_id] = {
                "count": 0,
                "temperatures": [],
                "corrupted": 0
            }

        device_data[device_id]["corrupted"] += 1
        continue

    # Create device entry
    if device_id not in device_data:
        device_data[device_id] = {
            "count": 0,
            "temperatures": [],
            "corrupted": 0
```

```
            }

        device_data[device_id]["count"] += 1
        device_data[device_id]["temperatures"].append(
            temperature
        )

    return device_data

def display_results(device_data):

    if not device_data:
        print("No valid records found.")
        return

    print("\nSensor Summary:")

    for device_id in sorted(device_data):

        data = device_data[device_id]

        temperatures = data["temperatures"]

        if temperatures:
            minimum = min(temperatures)
            maximum = max(temperatures)
            average = sum(temperatures) / len(temperatures)

            print(
                f"{device_id} "
                f"count={data['count']} "
                f"min={minimum:g} "
                f"max={maximum:g} "
                f"avg={average:.2f} "
                f"corrupted={data['corrupted']}"
            )
        else:
            print(
                f"{device_id} "
                f"count=0 "
                f"min=N/A "
                f"max=N/A "
                f"avg=N/A "
                f"corrupted={data['corrupted']}"
            )

def main():

    print("Generator-Based JSON ETL Pipeline")

    file_path = input(
        "Enter path of JSONL file: "
    ).strip()

    if file_path == "":
        print("File path cannot be empty.")
        return
```

```
    if not os.path.isfile(file_path):
        print("The specified file does not exist.")
        return

    device_data = process_records(file_path)

    display_results(device_data)

main()
```

Output :

```
Generator-Based JSON ETL Pipeline

Enter path of JSONL file: sensors.jsonl

Sensor Summary:
D1 count=1 min=30 max=30 avg=30.00 corrupted=1
D2 count=2 min=28 max=32 avg=30.00 corrupted=0
```

8. Matrix Path Optimizer with Dynamic Programming

```
MOD = 1000000007
NEGATIVE_INFINITY = float("-inf")

def read_matrix(n, m):
    matrix = []

    print("Enter matrix rows:")

    for i in range(n):
        while True:
            row_input = input(f"Row {i + 1}: ").strip()

            # Normal input: 1 2 3 X 5
            values = row_input.split()

            # Support assignment sample format like 123 / 4X6 / 789
            if len(values) == 1 and len(values[0]) == m:
                values = list(values[0])

            if len(values) != m:
                print(f"Please enter exactly {m} values.")
                continue

            row = []

            valid = True

            for value in values:

                if value.upper() == "X":
                    row.append(None)

                else:
                    try:
                        row.append(int(value))
                    except ValueError:
                        print(
                            "Invalid value. Use integers or X for blocked cells."
                        )
                        valid = False
                        break

            if valid:
                matrix.append(row)
                break

    return matrix

def find_maximum_path(matrix, n, m):

    # dp[j] stores the best score reaching current row, column j
    dp = [NEGATIVE_INFINITY] * m

    # ways[j] stores number of maximum-score paths
    ways = [0] * m
```

```
for i in range(n):

    new_dp = [NEGATIVE_INFINITY] * m
    new_ways = [0] * m

    for j in range(m):

        # Skip blocked cells
        if matrix[i][j] is None:
            continue

        # Starting cell
        if i == 0 and j == 0:
            new_dp[j] = matrix[i][j]
            new_ways[j] = 1
            continue

        candidates = []

        # From top
        if i > 0 and dp[j] != NEGATIVE_INFINITY:
            candidates.append(
                (dp[j], ways[j])
            )

        # From left
        if j > 0 and new_dp[j - 1] != NEGATIVE_INFINITY:
            candidates.append(
                (new_dp[j - 1], new_ways[j - 1])
            )

        # From diagonal
        if i > 0 and j > 0 and dp[j - 1] != NEGATIVE_INFINITY:
            candidates.append(
                (dp[j - 1], ways[j - 1])
            )

        if not candidates:
            continue

        best_previous_score = max(
            score for score, count in candidates
        )

        total_ways = 0

        for score, count in candidates:
            if score == best_previous_score:
                total_ways = (
                    total_ways + count
                ) % MOD

        new_dp[j] = (
            best_previous_score + matrix[i][j]
        )
```

```
                new_ways[j] = total_ways

            dp = new_dp
            ways = new_ways

        if dp[m - 1] == NEGATIVE_INFINITY:
            return None

        return dp[m - 1], ways[m - 1]

def main():

    print("Matrix Path Optimizer using Dynamic Programming")

    try:
        first_line = input(
            "Enter number of rows and columns: "
        ).split()

        if len(first_line) != 2:
            print("Please enter exactly two values.")
            return

        n = int(first_line[0])
        m = int(first_line[1])

        if n <= 0 or m <= 0:
            print("Rows and columns must be positive.")
            return

    except ValueError:
        print("Invalid input. Enter integers only.")
        return

    matrix = read_matrix(n, m)

    result = find_maximum_path(
        matrix,
        n,
        m
    )

    print("\nOutput:")

    if result is None:
        print("IMPOSSIBLE")
    else:
        maximum_score, number_of_paths = result

        print(
            maximum_score,
            number_of_paths
        )

main()
```

Output :

```
Matrix Path Optimizer using Dynamic Programming

Enter number of rows and columns: 3 3
Enter matrix rows:
Row 1: 1 2 3
Row 2: 4 X 6
Row 3: 7 8 9

Output:
29 1
```

9. OOP Inventory Merger with Operator Overloading

```
class Product:
    def __init__(self, product_id, name, stock, purchase_price, selling_price):
        self.product_id = product_id
        self.name = name
        self.stock = stock
        self.purchase_price = purchase_price
        self.selling_price = selling_price

    def total_value(self):
        return self.stock * self.purchase_price

class Inventory:
    def __init__(self, name):
        self.name = name
        self.products = {}

    def add_product(self, product):
        self.products[product.product_id] = product

    def delete_product(self, product_id):
        if product_id in self.products:
            del self.products[product_id]
            return True
        return False

    def update_stock(self, product_id, new_stock):
        if product_id in self.products:
            self.products[product_id].stock = new_stock
            return True
        return False

    def total_value(self):
        total = 0

        for product in self.products.values():
            total += product.total_value()

        return total

    def compare(self, other):
        value1 = self.total_value()
        value2 = other.total_value()

        if value1 > value2:
            return f"{self.name} has higher total value."
        elif value2 > value1:
            return f"{other.name} has higher total value."
        else:
            return "Both inventories have equal total value."

    # Operator overloading
    def __add__(self, other):
        merged = Inventory(
            self.name + " + " + other.name
        )
```

```
        # Add products from first inventory
        for product in self.products.values():
            merged.products[product.product_id] = Product(
                product.product_id,
                product.name,
                product.stock,
                product.purchase_price,
                product.selling_price
            )

        # Merge products from second inventory
        for product in other.products.values():

            if product.product_id in merged.products:

                existing = merged.products[product.product_id]

                # Combine stock
                existing.stock += product.stock

                # Keep lower purchase price
                existing.purchase_price = min(
                    existing.purchase_price,
                    product.purchase_price
                )

                # Keep higher selling price
                existing.selling_price = max(
                    existing.selling_price,
                    product.selling_price
                )

            else:
                merged.products[product.product_id] = Product(
                    product.product_id,
                    product.name,
                    product.stock,
                    product.purchase_price,
                    product.selling_price
                )

        return merged

    def display(self):
        if not self.products:
            print("Inventory is empty.")
            return

        for product_id in sorted(self.products):

            product = self.products[product_id]

            print(
                f"{product.product_id} "
                f"{product.name} "
                f"stock={product.stock} "
```

```
                f"purchase={product.purchase_price:g} "
                f"selling={product.selling_price:g}"
            )

def read_inventory(name):
    inventory = Inventory(name)

    try:
        count = int(
            input(f"Enter number of products in Inventory {name}: ")
        )

        if count < 0:
            print("Number of products cannot be negative.")
            return inventory

    except ValueError:
        print("Invalid number.")
        return inventory

    for i in range(count):

        print(f"\nProduct {i + 1}")

        product_id = input("Enter product ID: ").strip()
        product_name = input("Enter product name: ").strip()

        try:
            stock = int(input("Enter stock: "))
            purchase_price = float(
                input("Enter purchase price: ")
            )
            selling_price = float(
                input("Enter selling price: ")
            )

            if stock < 0 or purchase_price < 0 or selling_price < 0:
                print("Stock and prices cannot be negative.")
                continue

        except ValueError:
            print("Invalid numeric value.")
            continue

        product = Product(
            product_id,
            product_name,
            stock,
            purchase_price,
            selling_price
        )

        inventory.add_product(product)

    return inventory

def perform_operations(inventory_a, inventory_b):
```

```
while True:

    print("\nAvailable Operations")
    print("1. ADD")
    print("2. DELETE")
    print("3. UPDATE")
    print("4. MERGE")
    print("5. COMPARE")
    print("6. DISPLAY")
    print("7. EXIT")

    choice = input("Enter operation number: ").strip()

    if choice == "1":

        product_id = input("Enter product ID: ").strip()
        name = input("Enter product name: ").strip()

        try:
            stock = int(input("Enter stock: "))
            purchase = float(input("Enter purchase price: "))
            selling = float(input("Enter selling price: "))

            product = Product(
                product_id,
                name,
                stock,
                purchase,
                selling
            )

            inventory_a.add_product(product)

            print("ADD successful.")

        except ValueError:
            print("Invalid product details.")

    elif choice == "2":

        product_id = input("Enter product ID to delete: ").strip()

        if inventory_a.delete_product(product_id):
            print("DELETE successful.")
        else:
            print("Product not found.")

    elif choice == "3":

        product_id = input("Enter product ID: ").strip()

        try:
            new_stock = int(input("Enter new stock: "))

            if inventory_a.update_stock(
                product_id,
```

```
                    new_stock
                ):
                    print("UPDATE successful.")
                else:
                    print("Product not found.")

            except ValueError:
                print("Invalid stock value.")

        elif choice == "4":

            merged = inventory_a + inventory_b

            print("\nMerged Inventory:")
            merged.display()

            print(
                "Total valuation:",
                merged.total_value()
            )

        elif choice == "5":

            print(
                inventory_a.compare(inventory_b)
            )

            print(
                "Inventory A value:",
                inventory_a.total_value()
            )

            print(
                "Inventory B value:",
                inventory_b.total_value()
            )

        elif choice == "6":

            print("\nInventory A:")
            inventory_a.display()

            print("\nInventory B:")
            inventory_b.display()

        elif choice == "7":

            print("Program ended.")
            break

        else:
            print("Invalid operation.")

def main():

    print("OOP Inventory Merger")
```

```
    inventory_a = read_inventory("A")
    inventory_b = read_inventory("B")

    perform_operations(
        inventory_a,
        inventory_b
    )

main()
```

Output :

```
OOP Inventory Merger
Enter number of products in Inventory A: 1

Product 1
Enter product ID: P1
Enter product name: Pen
Enter stock: 10
Enter purchase price: 5
Enter selling price: 8
Enter number of products in Inventory B: 1

Product 1
Enter product ID: P1
Enter product name: Pen
Enter stock: 7
Enter purchase price: 4
Enter selling price: 9

Available Operations
1. ADD
2. DELETE
3. UPDATE
4. MERGE
5. COMPARE
6. DISPLAY
7. EXIT
Enter operation number: 4

Merged Inventory:
P1 Pen stock=17 purchase=4 selling=9
Total valuation: 68.0

Available Operations
1. ADD
2. DELETE
3. UPDATE
4. MERGE
5. COMPARE
6. DISPLAY
7. EXIT
Enter operation number: 7
Program ended.
patelvinshi@Patels-MacBook-Air assignment2 % 4
```

10. Python Data Visualization Dashboard Exporter

```
import csv
import os
import heapq
import matplotlib.pyplot as plt

def read_csv_data(file_path):
    """Read CSV file and validate its header."""
    if not os.path.exists(file_path):
        raise FileNotFoundError("Marks CSV file not found.")

    with open(file_path, "r", newline="", encoding="utf-8") as file:
        reader = csv.reader(file)
        header = next(reader, None)

        if header is None:
            raise ValueError("CSV file is empty.")

        if len(header) < 3:
            raise ValueError(
                "CSV must contain enrollment, name and at least one subject."
            )

        if header[0].strip().lower() != "enrollment":
            raise ValueError("First column must be 'enrollment'.")

        if header[1].strip().lower() != "name":
            raise ValueError("Second column must be 'name'.")

        return header

def calculate_subject_statistics(file_path, subject_names):
    """
    First pass:
    Calculate sum, count, minimum and maximum for every subject.
    Missing or invalid marks are ignored while calculating the mean.
    """
    statistics = {}

    for subject in subject_names:
        statistics[subject] = {
            "sum": 0.0,
            "count": 0,
            "minimum": 100.0,
            "maximum": 0.0
        }

    with open(file_path, "r", newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            for subject in subject_names:
                value = row.get(subject, "").strip()

                if value == "":
                    continue
```

```
                try:
                    mark = float(value)
                except ValueError:
                    continue

                if mark < 0 or mark > 100:
                    continue

                statistics[subject]["sum"] += mark
                statistics[subject]["count"] += 1

                if mark < statistics[subject]["minimum"]:
                    statistics[subject]["minimum"] = mark

                if mark > statistics[subject]["maximum"]:
                    statistics[subject]["maximum"] = mark

    # Calculate average for every subject.
    for subject in subject_names:
        stats = statistics[subject]

        if stats["count"] > 0:
            stats["average"] = stats["sum"] / stats["count"]
        else:
            stats["average"] = 0

            # No valid marks were found.
            stats["minimum"] = 0
            stats["maximum"] = 0

    return statistics

def get_clean_mark(value, subject_average):
    """Return a valid mark or replace missing/invalid mark with subject average."""
    value = value.strip()

    if value == "":
        return subject_average

    try:
        mark = float(value)
    except ValueError:
        return subject_average

    if mark < 0 or mark > 100:
        return subject_average

    return mark

def get_grade(average):
    """Assign a grade based on the student's average."""
    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
```

```
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    else:
        return "F"

def create_cleaned_csv(file_path, output_folder, subject_names, statistics):
    """
    Second pass:
    Clean missing values and create cleaned_marks.csv.
    Also calculate grade distribution and top performers.
    """
    cleaned_file = os.path.join(output_folder, "cleaned_marks.csv")

    grade_distribution = {
        "A+": 0,
        "A": 0,
        "B": 0,
        "C": 0,
        "D": 0,
        "F": 0
    }

    # Keep only top 5 students in memory.
    top_students = []

    with open(file_path, "r", newline="", encoding="utf-8") as input_file:
        reader = csv.DictReader(input_file)

        with open(
            cleaned_file,
            "w",
            newline="",
            encoding="utf-8"
        ) as output_file:

            fieldnames = (
                ["enrollment", "name"]
                + subject_names
                + ["Total", "Average", "Grade"]
            )

            writer = csv.DictWriter(
                output_file,
                fieldnames=fieldnames
            )

            writer.writeheader()

            for row in reader:
                cleaned_row = {
                    "enrollment": row.get("enrollment", "").strip(),
                    "name": row.get("name", "").strip()
                }
```

```
            total = 0.0

            for subject in subject_names:
                average = statistics[subject]["average"]

                mark = get_clean_mark(
                    row.get(subject, ""),
                    average
                )

                cleaned_row[subject] = f"{mark:.2f}"
                total += mark

            student_average = total / len(subject_names)
            grade = get_grade(student_average)

            cleaned_row["Total"] = f"{total:.2f}"
            cleaned_row["Average"] = f"{student_average:.2f}"
            cleaned_row["Grade"] = grade

            writer.writerow(cleaned_row)

            grade_distribution[grade] += 1

            student_info = (
                student_average,
                cleaned_row["enrollment"],
                cleaned_row["name"]
            )

            # Maintain top 5 students.
            if len(top_students) < 5:
                heapq.heappush(top_students, student_info)
            else:
                if student_info > top_students[0]:
                    heapq.heapreplace(top_students, student_info)

    top_students.sort(reverse=True)

    return cleaned_file, grade_distribution, top_students

def create_summary_csv(output_folder, statistics):
    """Create subject-wise summary CSV."""
    summary_file = os.path.join(output_folder, "summary.csv")

    with open(
        summary_file,
        "w",
        newline="",
        encoding="utf-8"
    ) as file:

        writer = csv.writer(file)

        writer.writerow([
            "Subject",
            "Average",
```

```
            "Minimum",
            "Maximum",
            "Valid Marks"
        ])

        for subject, stats in statistics.items():
            writer.writerow([
                subject,
                f"{stats['average']:.2f}",
                f"{stats['minimum']:.2f}",
                f"{stats['maximum']:.2f}",
                stats["count"]
            ])

    return summary_file

def create_grade_distribution_chart(grade_distribution, output_folder):
    """Create grade distribution chart."""
    grades = ["A+", "A", "B", "C", "D", "F"]
    counts = [grade_distribution[grade] for grade in grades]

    plt.figure(figsize=(8, 5))
    plt.bar(grades, counts)

    plt.title("Grade Distribution")
    plt.xlabel("Grade")
    plt.ylabel("Number of Students")

    plt.tight_layout()

    file_path = os.path.join(
        output_folder,
        "grade_distribution.png"
    )

    plt.savefig(file_path)
    plt.close()

    return file_path

def create_subject_average_chart(statistics, output_folder):
    """Create subject-wise average comparison chart."""
    subjects = list(statistics.keys())

    averages = [
        statistics[subject]["average"]
        for subject in subjects
    ]

    plt.figure(figsize=(8, 5))
    plt.bar(subjects, averages)

    plt.title("Subject-wise Average Comparison")
    plt.xlabel("Subject")
    plt.ylabel("Average Marks")
    plt.ylim(0, 100)
```

```
    plt.tight_layout()

    file_path = os.path.join(
        output_folder,
        "subject_average.png"
    )

    plt.savefig(file_path)
    plt.close()

    return file_path

def create_top_performer_chart(top_students, output_folder):
    """Create top performer chart."""
    names = [
        student[2]
        for student in top_students
    ]

    averages = [
        student[0]
        for student in top_students
    ]

    plt.figure(figsize=(8, 5))
    plt.bar(names, averages)

    plt.title("Top Performers")
    plt.xlabel("Student")
    plt.ylabel("Average Marks")
    plt.ylim(0, 100)

    plt.tight_layout()

    file_path = os.path.join(
        output_folder,
        "top_performers.png"
    )

    plt.savefig(file_path)
    plt.close()

    return file_path

def main():
    print("Python Data Visualization Dashboard Exporter")

    marks_file = input(
        "Enter path of marks CSV: "
    ).strip()

    output_folder = input(
        "Enter output folder path: "
    ).strip()

    if not marks_file:
        print("Error: CSV path cannot be empty.")
```

```
        return

if not output_folder:
        print("Error: Output folder cannot be empty.")
        return

try:
        os.makedirs(output_folder, exist_ok=True)

        # Read and validate CSV header.
        header = read_csv_data(marks_file)

        # First two columns are enrollment and name.
        subject_names = header[2:]

        if len(subject_names) > 20:
                print("Error: Maximum 20 subjects are allowed.")
                return

        # Calculate subject statistics.
        statistics = calculate_subject_statistics(
                marks_file,
                subject_names
        )

        # Create cleaned CSV and calculate performers/grades.
        cleaned_file, grade_distribution, top_students = (
                create_cleaned_csv(
                        marks_file,
                        output_folder,
                        subject_names,
                        statistics
                )
        )

        # Create subject summary.
        summary_file = create_summary_csv(
                output_folder,
                statistics
        )

        # Create three required charts.
        grade_chart = create_grade_distribution_chart(
                grade_distribution,
                output_folder
        )

        subject_chart = create_subject_average_chart(
                statistics,
                output_folder
        )

        top_chart = create_top_performer_chart(
                top_students,
                output_folder
        )
```

```
        print("\nProcessing completed successfully.")

        print("\nSubject-wise Statistics:")
        for subject in subject_names:
            stats = statistics[subject]

            print(
                f"{subject}: "
                f"Average={stats['average']:.2f}, "
                f"Minimum={stats['minimum']:.2f}, "
                f"Maximum={stats['maximum']:.2f}"
            )

        print("\nTop Performers:")
        for position, student in enumerate(top_students, start=1):
            print(
                f"{position}. "
                f"{student[2]} "
                f"({student[0]:.2f})"
            )

        print("\nOutput Files:")
        print(cleaned_file)
        print(summary_file)
        print(grade_chart)
        print(subject_chart)
        print(top_chart)

    except Exception as error:
        print("\nError:", error)

if __name__ == "__main__":
    main()
```

Output :

```
Python Data Visualization Dashboard Exporter
Enter path of marks CSV: marks.csv
Enter output folder path: dashboard_output

Processing completed successfully.

Subject-wise Statistics:
Python: Average=82.00, Minimum=65.00, Maximum=92.00
DBMS: Average=86.25, Minimum=72.00, Maximum=94.00
OS: Average=84.80, Minimum=68.00, Maximum=95.00

Top Performers:
1. Neha (91.67)
2. Dev (92.00)
3. Asha (87.67)
4. Rahul (82.08)
5. Priya (68.33)

Output Files:
dashboard_output/cleaned_marks.csv
dashboard_output/summary.csv
dashboard_output/grade_distribution.png
dashboard_output/subject_average.png
dashboard_output/top_performers.png
```